

KneeVision



Team S.T.A.R Labs

Agenda

01



SPRINT 1
RECAP

- Research Team
- Improvements
- Project Description
- Team Working Agreement
- Personas

02



PROJECT
OVERVIEW

- MVP
- Languages and Tools
- Algorithms
- Diagrams

03



PRODUCT
BACKLOG

- User Stories
- Sprint 2 Stories
- Test Cases

04



TEAM
METRICS

- Team Velocity
- Burndown Charts
- Completed/Committed Ratio

05



PRODUCT

- Sprint 2 Retrospective
- Sprint 3 Planning
- API
- Live Application Demo

Our Research Team



Daniel Fox
**Team Leader/
Developer**



Manohar Killamsetti
**Scrum
Master/Developer**



Siming Li
Developer/Tester



Sarvesh Shah
Cloud Developer



Swatej Goud
Backend Developer



Deep Patel
ML Developer

Improvements

- Added more information for the model pipeline overview including diagrams, datasets used, and generating synthetic data (slides 21 – 25) 
- Updated the ER diagram (slide 28) 
- Color-coded the sprints in the product/sprint 2 backlog (slides 33 – 37) 
- Added charts instead of screenshots for the project schedule, product backlog, and test cases sections (slides 38 – 51) 
- Added charts to Team Velocity and Completed/Committed Ratio slides (slide 52 + 54) 

Problem Statement

Knee X-ray imaging is commonly used to evaluate osteoarthritis, yet interpreting radiographic signs of disease severity can be subjective and inconsistent due to subtle visual differences and reliance on manual assessment. While advanced imaging techniques such as MRI provide greater detail, they are expensive and less accessible. This creates a need for scalable and consistent analysis methods that can assist in assessing knee osteoarthritis using widely available X-ray images.

Project Definition

- A web-based platform that uses deep learning to analyze knee X-ray images for knee osteoarthritis severity assessment.
- Intended for individuals, researchers, and clinicians seeking consistent and scalable analysis of knee X-ray data.
- Leverages publicly available datasets and established convolutional neural network (CNN) models.
- Designed as an AI-assisted decision-support and research tool, not a diagnostic system.
- Benefits include improved consistency, scalability, and reduced subjectivity in manual X-ray interpretation.

Project Description

Project Name:	KneeVision
Team:	S.T.A.R. Labs
Project Description:	KneeVision is a web-based platform that uses machine learning to analyze knee X-ray images, enabling faster and more accessible diagnostic insights. For researchers and medical professionals who upload knee X-ray images the KneeVision app is a web-based application that uses machine learning to analyze knee X-rays and assist in detecting potential conditions that provides faster, data-driven insights to support diagnosis and decision-making unlike traditional workflows that rely solely on manual review by specialists and can be time-consuming our application improves accessibility and efficiency by enabling quick image analysis and centralized result management
Benefit Outcomes:	• Faster turnaround time for preliminary image analysis • Improved accessibility to diagnostic insights • Centralized storage of user-specific medical images • Enhanced workflow for reviewing and managing X-ray data • Scalable foundation for integrating advanced AI/ML models
Github Link:	https://github.com/htmw/STARLABS

Team Agreement: Collaboration Standards

Meeting Pattern

Synchronous sessions twice weekly (Sun/Tue).

Mandated Agenda

documents for every session.

Continuous async sync via WhatsApp/Discord.

Decision Making

Majority-vote consensus based on researched evidence. Inclusive voicing of concerns during all discussion phases.

Technical Workflow

No direct pushes to Main. Isolated feature branches only.

Peer Approval. Every PR requires at least one secondary reviewer.

Jira Hygiene. Mandatory linking of work to Jira tracker IDs.

Blockers. Resolve locally & document attempts before escalation.

Team Agreement: Definition of “Done”

Category	Standard for Completion
Execution	Code runs without syntactical or runtime errors.
Compliance	Work meets all Industry Standards.
Verification	Work reviewed by multiple members & verified on multiple systems.
Documentation	All necessary docs tagged or updated along with code push.
Testing	All pre-defined test scenarios must be passed.

Persona - 1: The Practitioner

ORTHOPEDIC SURGEON | URBAN TEACHING HOSPITAL

Dr. Sarah Chen, 42

An expert in orthopedic medicine with 15 years of experience. Sarah operates in high-volume clinics where diagnostic speed is paramount to patient throughput and surgical triage.

KEY GOALS

- Rapid knee X-ray triage
- Reduce initial assessment time
- Validation for borderline cases
- Fatigue from patient volume
- Reader reporting variability
- Limited case analysis time



Persona - 2: The Research Coordinator

CLINICAL RESEARCH COORDINATOR

Alex Rodriguez, 29

A data-centric professional managing multi-site studies. Alex focuses on standardization, data integrity, and inter-rater reliability across large datasets.

STRATEGIC GOALS

Standardize severity grading	Efficient batch processing
Structured data exporting	Ensure rater reliability

OPERATIONAL PAINS

Manual grading	Lengthy training cycles
inconsistency	Audit trail requirements
Historical data re-analysis	



Persona - 3: The Learner



3RD YEAR MEDICAL STUDENT

Maya Patel, 24

A digital native in her clinical rotations. Maya uses the platform as a pedagogical tool to build pattern recognition skills and diagnostic confidence before her residency.

LEARNING GOALS

- Identify radiographic signs
- Immediate feedback practice
- Grade/Clinical correlation

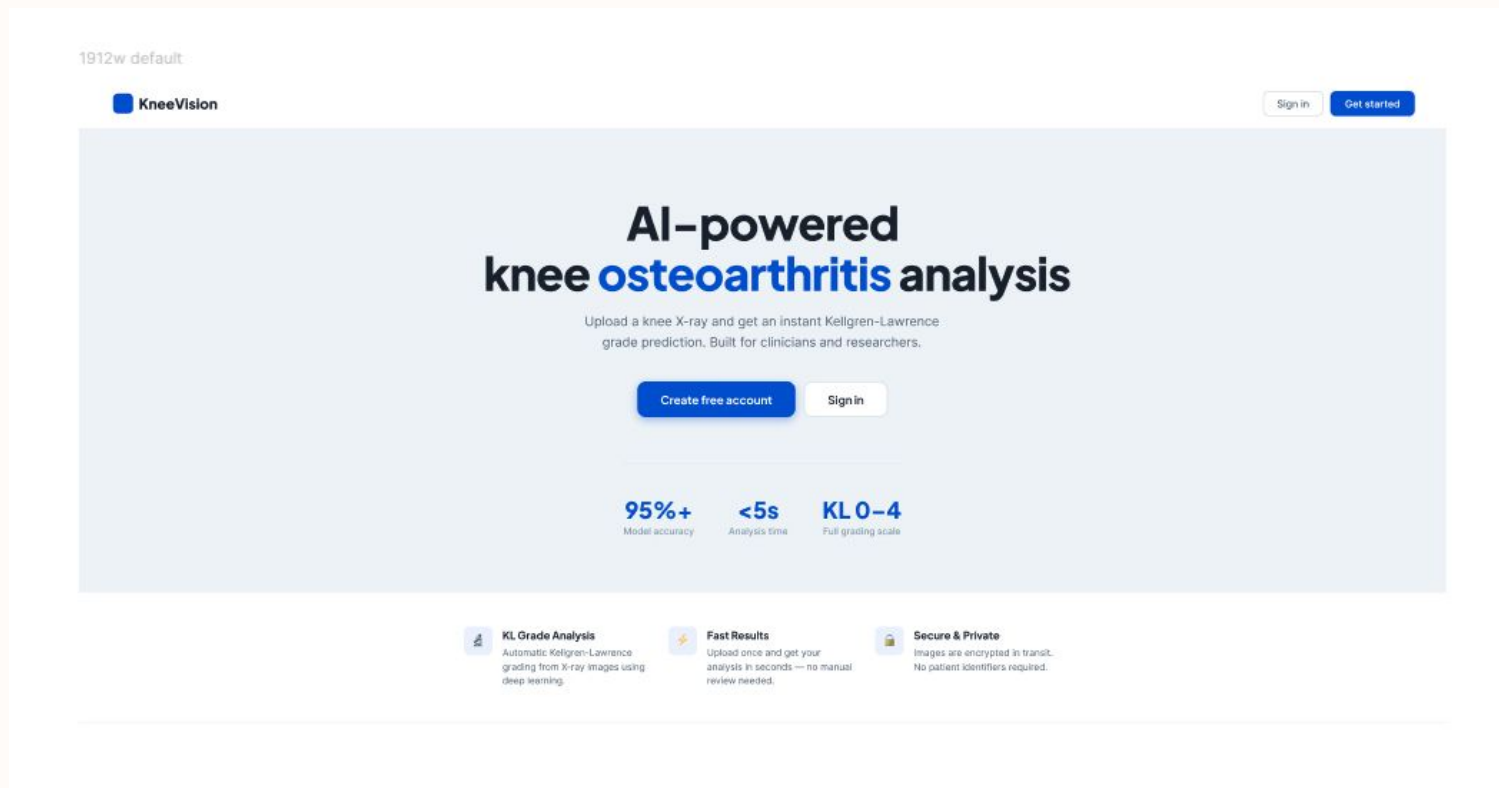
ACCESS OBSTACLES

- Limited diverse case access
- Assessment uncertainty
- Limited attending time

What will our MVP be?

- Web-based interface
- Image preprocessing pipeline
- Deep learning model
- Severity assessment output
- Basic result visualization
- Mobile-responsive UI

Project Design (Figma)



Tools & Technologies

Languages:  

Frontend:  

Backend:  

ML/DL Libraries: 

Database:  

Cloud: 

Tools:    

ML Inference: 

Misc. Libraries:  

FRONTEND

React · Vite · Tailwind CSS · TypeScript



React 18

Component-based UI

- Open-source JavaScript library for interactive UIs
- Reusable component architecture for maintainability
- State management with useState and useEffect hooks
- Pages: Landing, Login, Register, Upload, Gallery



Vite

Build toolchain

- Lightning-fast HMR and dev server startup
- Instant compilation during development
- Optimised production builds with tree-shaking
- Configured with Vite proxy to backend



Tailwind CSS

Utility-first styling

- Direct utility approach replaces custom CSS files
- Consistent design tokens across all components
- Responsive layouts with mobile-first breakpoints
- Dark-mode ready with zero extra config



TypeScript

Type-safe JavaScript

- Static typing catches errors at compile time
- Typed props and API response interfaces
- Improves IDE autocomplete and refactoring
- Compiled to JavaScript for browser execution

BACKEND

Node.js · Express.js · JWT · bcrypt



Node.js

Runtime environment

JavaScript runtime built on Chrome's V8 engine. Enables server-side JS execution with non-blocking I/O for high concurrency — perfect for handling multiple concurrent upload and analysis requests.



Express.js

Web framework

Minimal and flexible Node.js web framework. Handles all REST API routes: auth (register/login), image presign, raw file upload, image metadata registration, and image retrieval with JWT middleware.



JWT Auth

Stateless authentication

JSON Web Tokens secure every protected route via the requireAuth middleware. Tokens are signed with a secret, expire in 1 hour, and carry userId + email so any route knows who made the request.



bcrypt

Password hashing

Industry-standard adaptive hashing algorithm. Passwords are never stored in plain text — bcrypt hashes with salt rounds of 10, making brute-force attacks computationally infeasible.

DATABASE & CLOUD

MongoDB · AWS · Local filesystem



MongoDB

NoSQL document database

users collection

Stores email, hashed password, userId, timestamps

images collection

fileUrl, originalName, contentType, uploadedBy { userId, email }

predictions collection

imageId FK, klGrade, confidence, probabilities, modelVersion

Indexing

Unique index on users.email prevents duplicate accounts



AWS / Cloud

Cloud infrastructure (planned)

Amazon S3

Object storage for uploaded X-ray images at scale

EC2 / Lambda

Scalable compute for Express backend and ML inference

CloudFront

CDN for fast global delivery of static assets

Current dev

Local filesystem /uploads/ used during development

DEVOPS & TOOLS

Docker · GitHub · Jira · VS Code · Jest · Vitest



Docker

Containerises the Express backend and ML service so the environment is identical across dev, staging, and production machines.



GitHub

Version control and collaboration. Feature branches, pull requests, and code review. CI/CD pipeline triggers test suite on every push.



Jira

Agile project management. SCRUM board tracks sprints (SCRUM-18 register, SCRUM-19 login, SCRUM-20 image upload). Backlog prioritised by story points.



VS Code

Primary IDE with TypeScript, ESLint, and Prettier extensions. Integrated terminal runs both Vite dev server and Express backend simultaneously.



Jest + Vitest

Backend unit and integration tests with Jest (86 tests, ~89% coverage). Frontend component tests with Vitest + React Testing Library.



ESLint + Prettier

Code quality enforced at save. ESLint catches bugs, Prettier formats consistently. Husky pre-commit hooks block bad commits.

Model Pipeline Explanation

Dataset: Mendeley Dataset (8260 Images)

Model Plan:

Cleaning Dataset (Manually deleting the the defective images and cropping to ROI [Region of Interest])

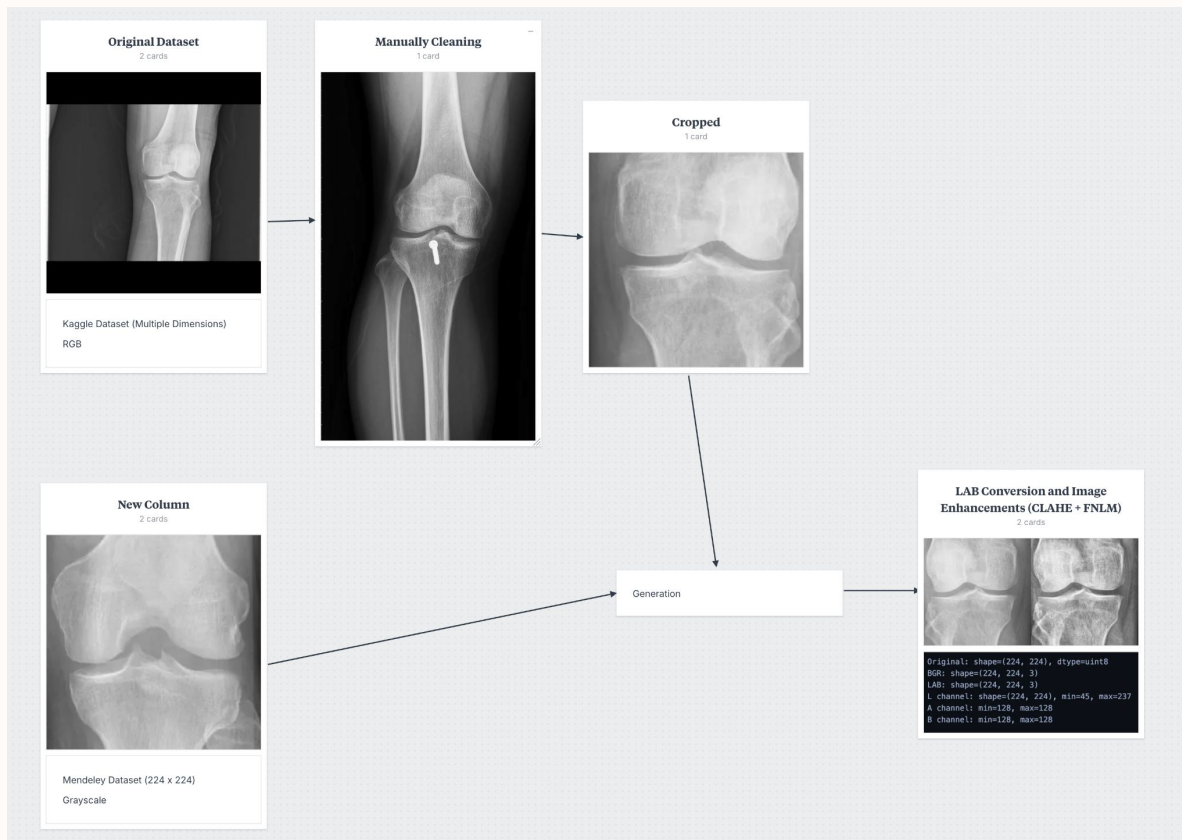
Generating synthetic data using DCGAN (Deep Convolutional Generative Adversarial Network)

Preprocessing (Enhancing images using Adaptive Histogram Equalization [AHE])

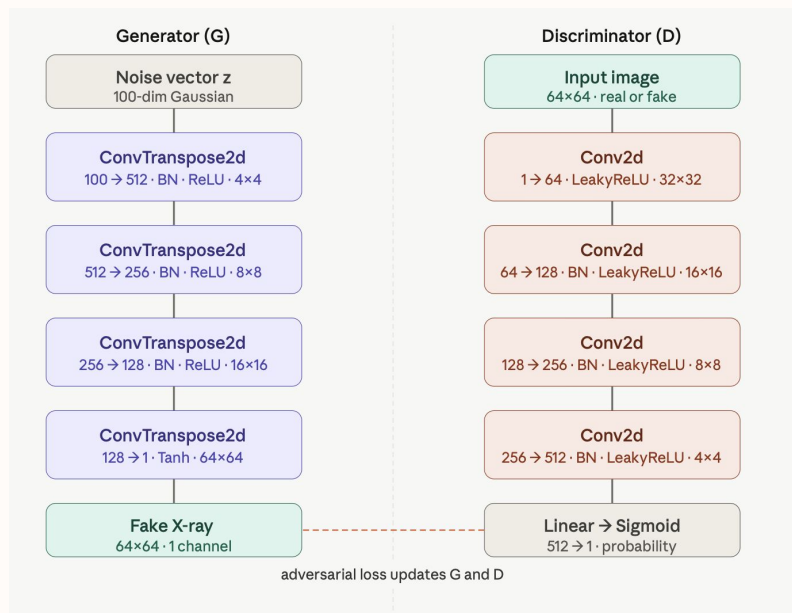
Using Pre-Trained Models

Metrics Evaluation

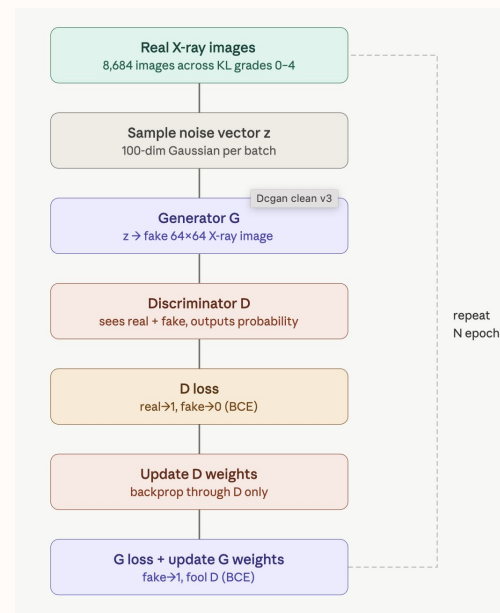
Model Pipeline - Dataset



Generating Synthetic Data - DCGAN

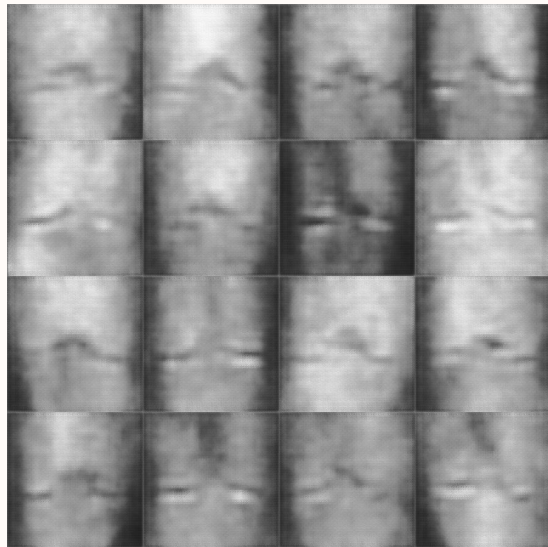


DCGAN Architecture

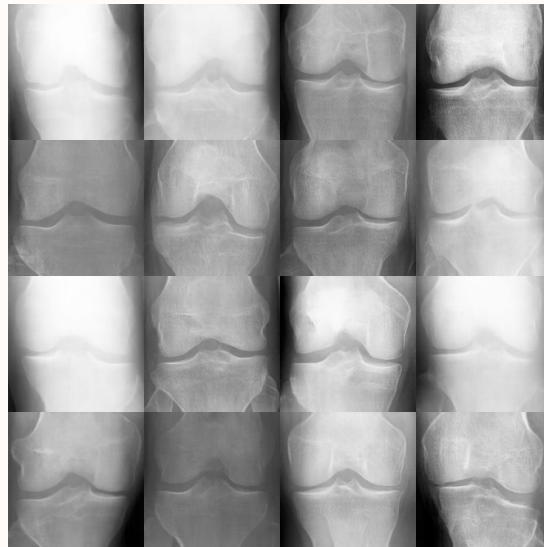


Generation Process

Generating Synthetic Data - DCGAN



Epoch 0



Epoch 200

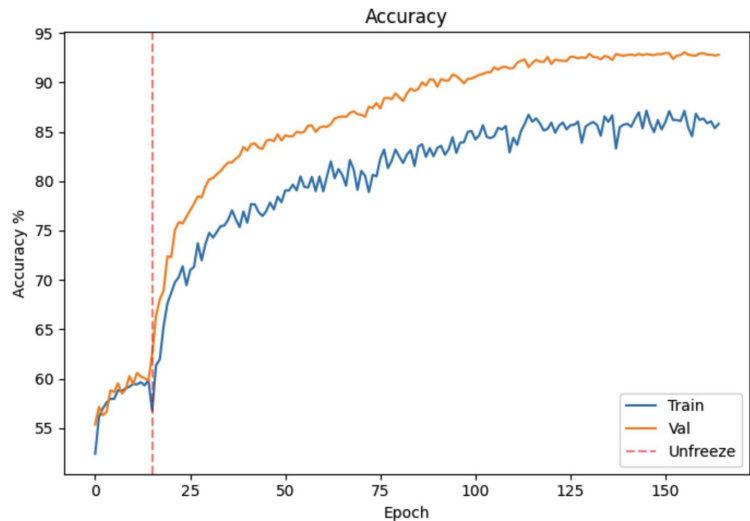
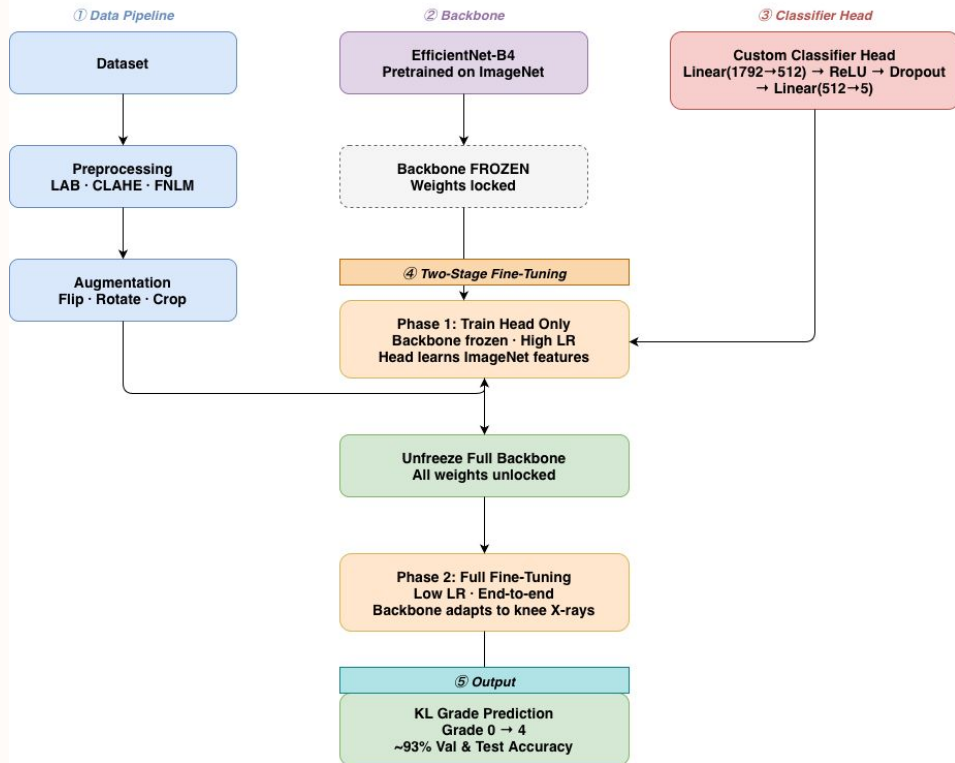


Generating Synthetic Data - DCGAN



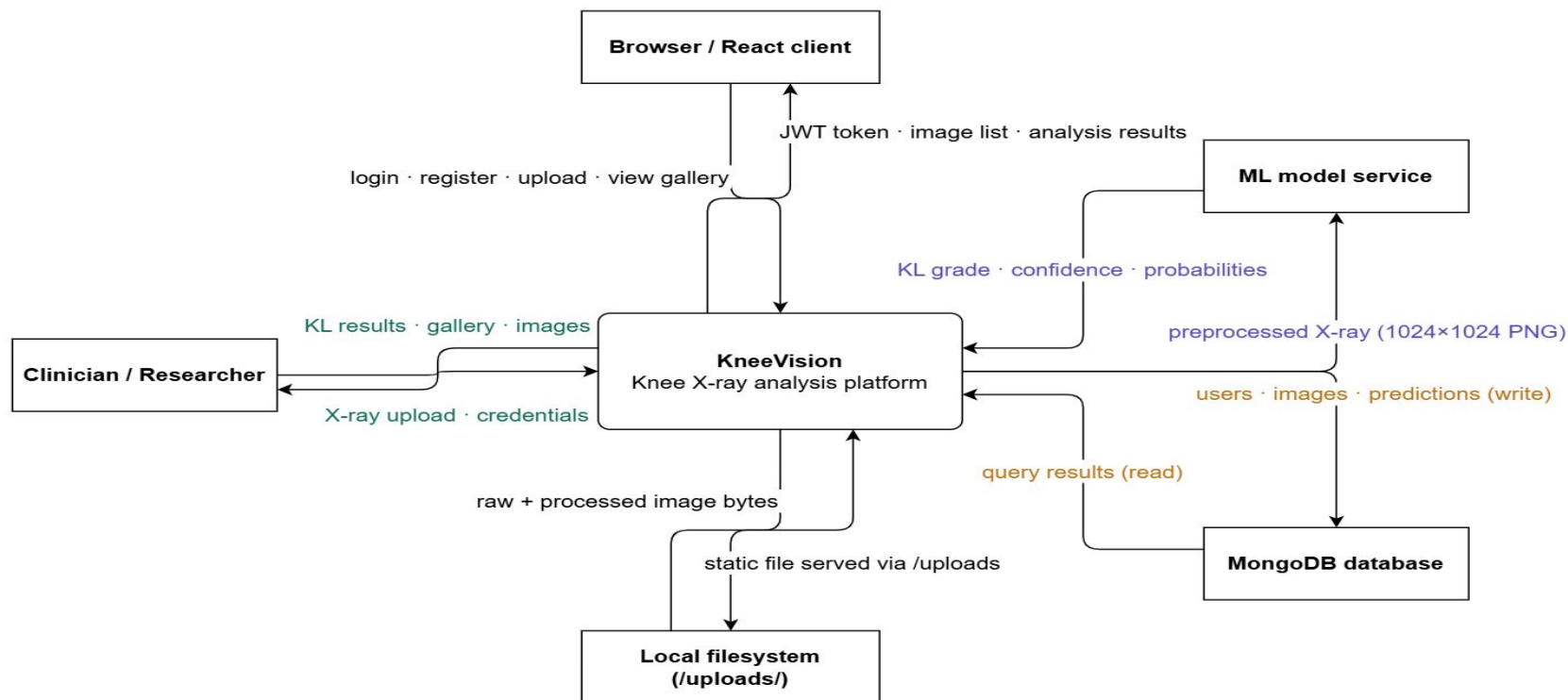
Model Overview

KneeVision — EfficientNet-B4 Training Process

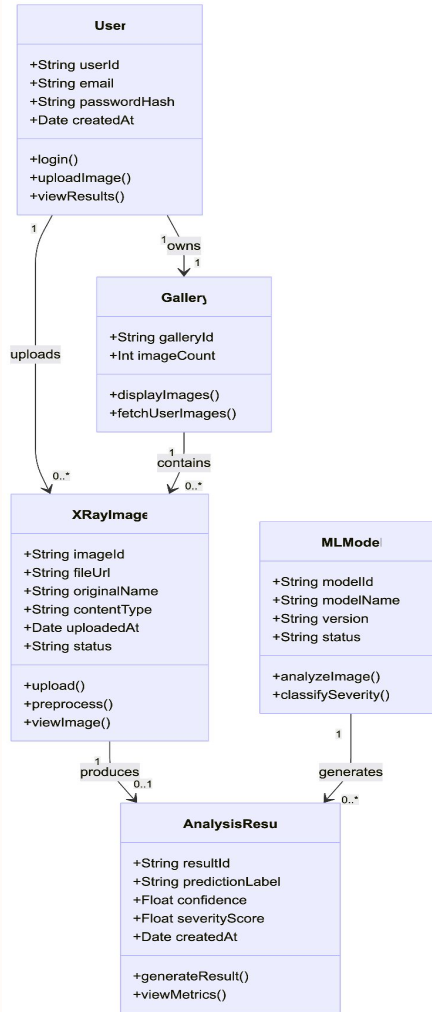


Context Diagram

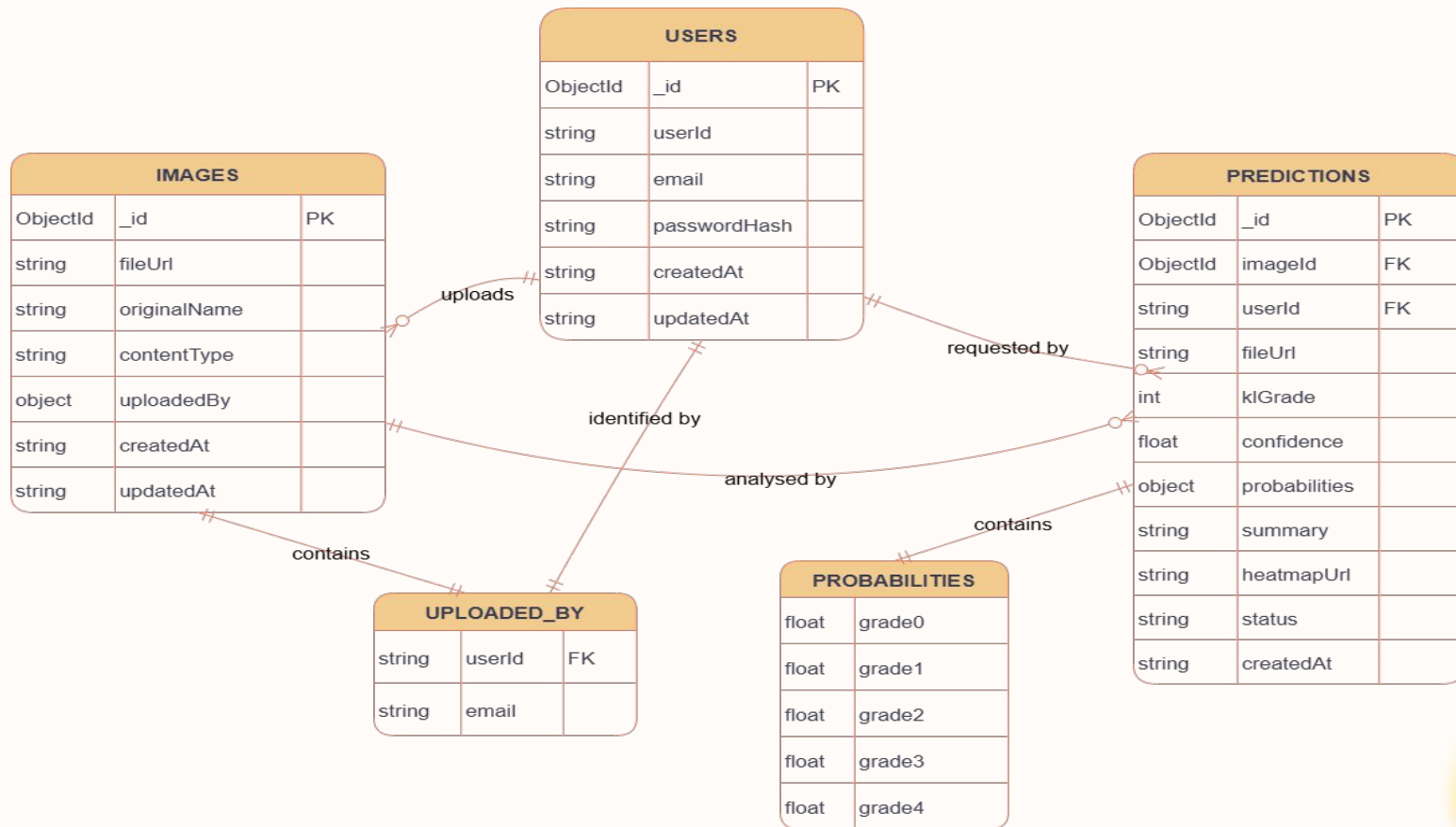
Context Diagram



Class Diagram

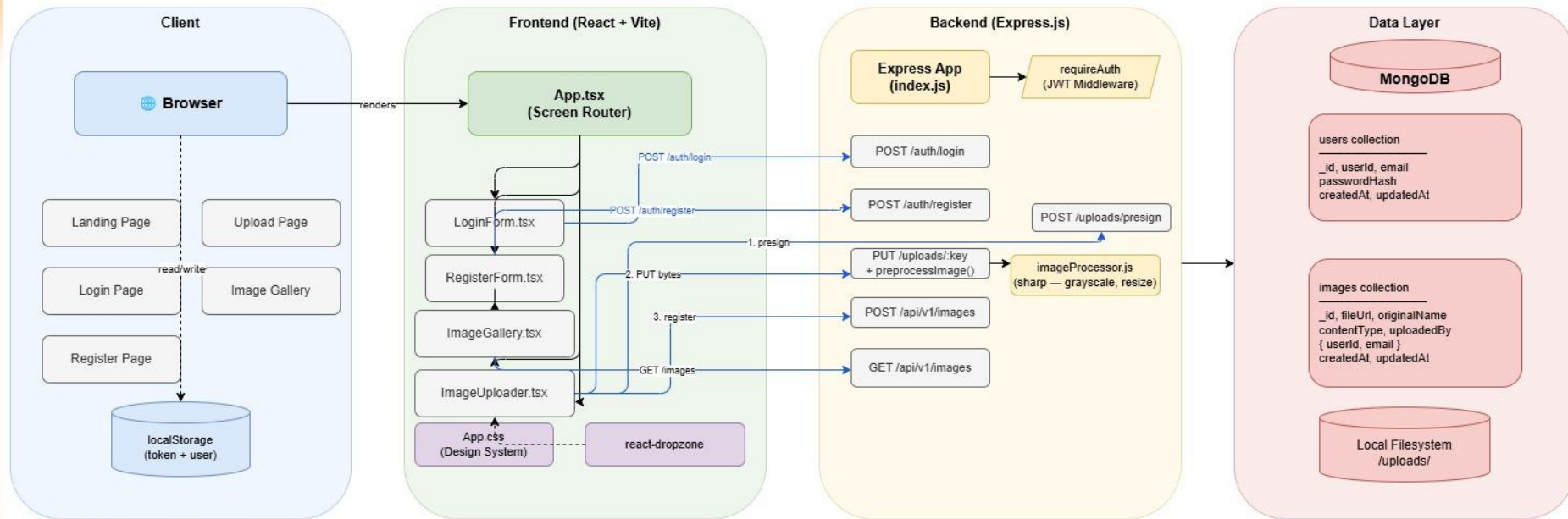


ER Diagram

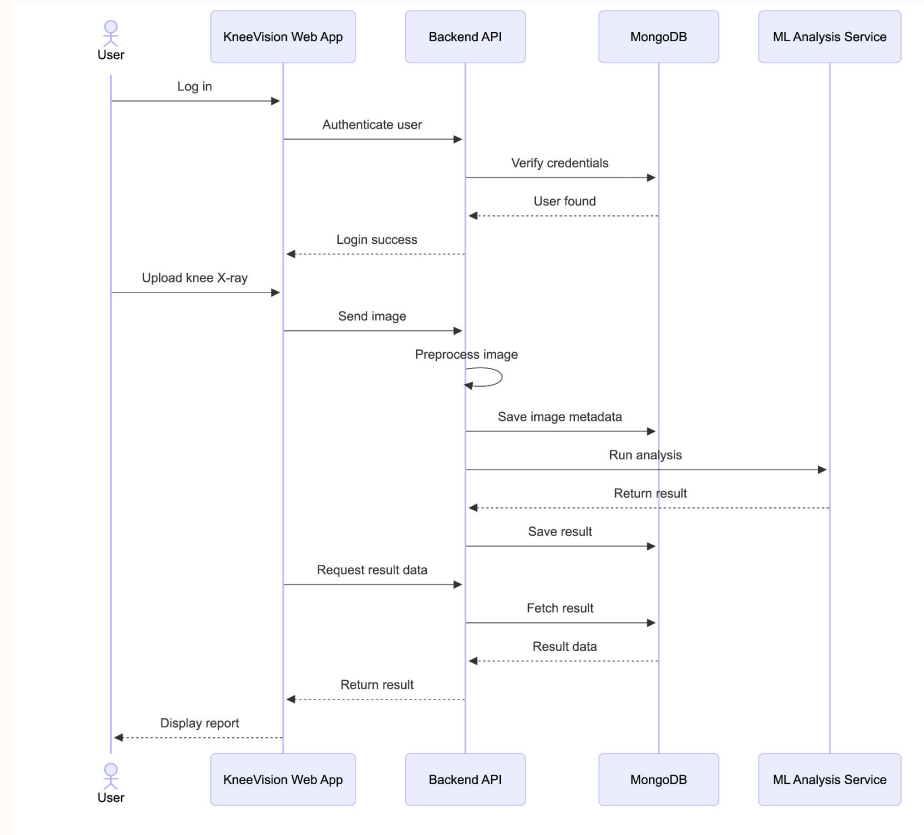


Architecture Diagram

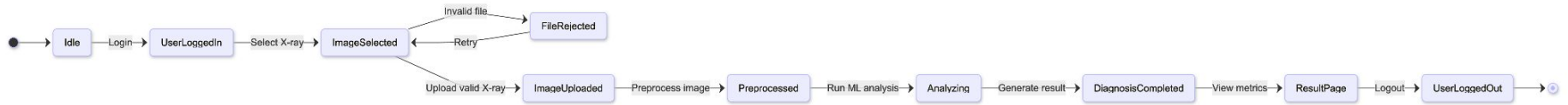
KneeVision — System Design



Sequence Diagram



State Diagram



Sprint 1 Recap

- Set up full-stack app (React + Vite + Node/Express)
- Implemented JWT authentication (login & registration)
- Built core UI screens (landing, auth, upload)
- Developed image upload + storage pipeline (MongoDB)
- Added preprocessing for ML readiness (resize/normalize)
- Created gallery view for uploaded images

PRODUCT BACKLOG - USER STORIES

Sprint 1

Sprint 2

Sprint 3



US_ID	User Stories	Acceptance Criteria	Feature	Story Points
US_01	As an individual user, I want to upload knee X-ray images so that I can receive an AI-based assessment of osteoarthritis severity.	The patient can access an interface to upload an image. The patient can select and submit an image from device. The system accepts image files in specified formats: JPEG, JPG, PNG, DICOM. The uploaded image is stored and associated with the patient's case. A confirmation message is displayed upon successful upload.	Image Upload	13
US_02	As a research coordinator, I want to upload multiple X-ray images at once so that I can analyze datasets efficiently.	The patient can upload multiple images (up to 3) to one case. The system accepts multiple images in specified formats (JPEG, JPG, PNG). All uploaded images are associated with the same patient case. The patient receives a confirmation message after uploading each image. The system displays a preview of all uploaded images for review before analysis.	Image Upload	8

PRODUCT BACKLOG - USER STORIES

US_ID	User Stories	Acceptance Criteria	Feature	Story Points
US_03	As an individual user, I want to register an account so that I can store my basic information and access personalised services.	The user can access a registration page requiring: Full name, Email address, Password creation. Post-registration, the user can log in and update their profile information.	User Management	2
US_04	As an individual user, I want to log in to my account so that I can manage my cases and view diagnoses.	The user accesses a login page requiring: Email address, Password. Upon successful login, the user is directed to their dashboard.	User Management	1
US_05	As a user, I want to view my past cases and track their statuses so that I can monitor my health progress.	The user can access a 'Case History' section after logging in. All past cases are listed in chronological order (most recent first). Each case displays: case ID, case open date, status. The system updates case statuses in real-time. Clicking on a case entry navigates the user to that case's report.	User Management	2
US_06	As an individual user, I want to implement JWT authentication so that my session is secure.	JWT tokens are generated on login and validated on every protected route. Tokens expire after a defined period and require re-authentication. Invalid or expired tokens return a 401 Unauthorized response.	User Management	3

PRODUCT BACKLOG - USER STORIES

US_ID	User Stories	Acceptance Criteria	Feature	Story Points
US_07	As an individual user, I want to receive an OA severity diagnosis so that I know the knee condition I am facing.	After submitting the image, the user receives a diagnosis. The diagnosis is presented as the KL grade (0–4) and severity label. The system provides the diagnosis within a reasonable time (< 1 min). The diagnosis is stored and associated with the user's case.	AI Model Integration	20
US_08	As an individual user, I want to receive the most accurate possible diagnosis so that I can trust the assessment provided.	The system continuously improves the model and algorithms for better diagnostic results. The diagnosis accuracy is optimised based on model refinements and external validation. The confidence level of the diagnosis should be high (e.g., 95%).	AI Model Integration	11
US_09	As an individual user, I want to receive a brief description of the diagnosed condition so that I can understand the issue I am facing.	After receiving the diagnosis, the user is provided with a brief description of the diagnosed condition. The description is written in clear, understandable language. The description includes common symptoms and causes.	AI Model Integration	2
US_10	As an individual user, I want to receive treatment recommendations so that I can take action towards treating my knee condition.	After receiving the diagnosis, the user is provided with treatment recommendations. The description is written in clear, understandable language. The description includes common symptoms and causes of the condition.	AI Model Integration	3

Sprint 2 Backlog

US_ID	User Stories	Acceptance Criteria	Feature
US_02	As a research coordinator, I want to upload multiple X-ray images at once so that I can analyze datasets efficiently.	The patient can upload multiple images (up to 3) to one case. The system accepts multiple images in specified formats (JPEG, JPG, PNG). All uploaded images are associated with the same patient case. The patient receives a confirmation message after uploading each image. The system displays a preview of all uploaded images for review before analysis.	Image Upload
US_08	As an individual user, I want to receive the most accurate possible diagnosis so that I can trust the assessment provided.	The system continuously improves the model and algorithms for better diagnostic results. The diagnosis accuracy is optimised based on model refinements and external validation. The confidence level of the diagnosis should be high (e.g., 95%).	AI Model Integration



Sprint 2 Backlog

US_ID	User Stories	Acceptance Criteria	Feature
US_09	As an individual user, I want to receive a brief description of the diagnosed condition so that I can understand the issue I am facing.	After receiving the diagnosis, the user is provided with a brief description of the diagnosed condition. The description is written in clear, understandable language. The description includes common symptoms and causes.	AI Model Integration
US_10	As an individual user, I want to receive treatment recommendations so that I can take action towards treating my knee condition.	After receiving the diagnosis, the user is provided with treatment recommendations. The description is written in clear, understandable language. The description includes common symptoms and causes of the condition.	AI Model Integration



Project Schedule - Sprint 2

Key	Issue Type	Parent	Summary	Status	Assignee	Sprint
SCRUM-91	Story	SCRUM-78	As an individual user, I want to use the platform on my phone so that I can access it from anywhere	Done		Sprint 2
SCRUM-53	Task	SCRUM-79	Technical paper - Implementation section	Done	Deep	Sprint 2
SCRUM-50	Task	SCRUM-79	Integration testing (upload -> predict -> display)	Done	Siming Li	Sprint 2
SCRUM-37	Task	SCRUM-79	Train model on Google Colab	Done	Deep	Sprint 2
SCRUM-38	Task	SCRUM-79	Evaluate model (target 75%+ accuracy)	Done	Deep	Sprint 2
SCRUM-39	Task	SCRUM-79	Save trained model (.pt format)	Done	Deep	Sprint 2
SCRUM-45	Task	SCRUM-79	Build results display page	Done	Daniel Fox	Sprint 2
SCRUM-92	Task	SCRUM-79	Create and train model for converting xray to unet images	Done	Sarvesh Shah	Sprint 2
SCRUM-93	Task	SCRUM-79	Feature extraction from unet images for training model	Done	Sarvesh Shah	Sprint 2
SCRUM-46	Task	SCRUM-79	Build results display page 1	Done	Siming Li	Sprint 2
SCRUM-44	Task	SCRUM-79	Store prediction results in database	Done	Manohar	Sprint 2
SCRUM-40	Task	SCRUM-79	Create predictions table in database	Done	Manohar	Sprint 2
SCRUM-54	Task	SCRUM-79	Create system architecture diagram	Done	Manohar	Sprint 2
SCRUM-34	Task	SCRUM-79	Update API documentation (Swagger)	Done	Manohar	Sprint 2

Project Schedule - Sprint 2

Key	Issue Type	Parent	Summary	Status	Assignee	Sprint
SCRUM-41	Task	SCRUM-79	Build model serving endpoint	Done	Swatej Goud Mella	Sprint 2
SCRUM-42	Task	SCRUM-79	Build /predict API endpoint	Done	Swatej Goud Mella	Sprint 2
SCRUM-48	Task	SCRUM-79	Create KL grade explanation modal	Done	Swatej Goud Mella	Sprint 2
SCRUM-49	Task	SCRUM-79	Add probability distribution visualization	Done	Swatej Goud Mella	Sprint 2
SCRUM-52	Task	SCRUM-79	Technical paper - Methodology section	Done	Deep	Sprint 2
SCRUM-63	Task	SCRUM-80	Cross-browser testing (Chrome, Firefox, Safari)	Done	Siming Li	Sprint 2
SCRUM-56	Task	SCRUM-80	Build analysis history page	Done	Daniel Fox	Sprint 2
SCRUM-57	Task	SCRUM-80	Add filter and sort functionality	Done	Siming Li	Sprint 2
SCRUM-55	Task	SCRUM-80	Document model training process	Done	Sarvesh Shah	Sprint 2



Project Schedule - Sprint 2 (Spill Over)

Key	Issue Type	Parent	Summary	Status	Assignee	Sprint
SCRUM-59	Task	SCRUM-80	Add educational tooltips throughout app	To Do	Swatej Goud Mella	Sprint 2
SCRUM-60	Task	SCRUM-80	Make UI responsive (mobile/tablet/desktop)	To Do	Daniel Fox	Sprint 2



Product Backlog - Outstanding

ID	Summary	Epic	Status	Priority
SCRUM-59	Add educational tooltips throughout app	MVP Completion & Deployment	TO DO	Low
SCRUM-60	Make UI responsive (mobile/tablet/desktop)	MVP Completion & Deployment	TO DO	High
SCRUM-33	Deploy to development environment	User Management & Image Upload	TO DO	Medium
SCRUM-43	Test prediction API (< 7 sec response)	AI Model Integration	TO DO	Medium
SCRUM-47	Show confidence score with progress bar	AI Model Integration	TO DO	Medium
SCRUM-61	Add error logging (backend + frontend)	MVP Completion & Deployment	TO DO	Medium
SCRUM-62	Conduct UAT with 3-5 external testers	MVP Completion & Deployment	TO DO	Medium
SCRUM-64	Performance testing (150 concurrent users)	MVP Completion & Deployment	TO DO	Medium
SCRUM-65	Security audit (OWASP top 10)	MVP Completion & Deployment	TO DO	Medium
SCRUM-66	Fix all critical and high-priority bugs	MVP Completion & Deployment	TO DO	Medium
SCRUM-67	Deploy frontend to Vercel/Netlify	MVP Completion & Deployment	TO DO	Medium
SCRUM-68	Deploy backend to Render/Railway	MVP Completion & Deployment	TO DO	Medium



Product Backlog - Outstanding

ID	Summary	Epic	Status	Priority
SCRUM-69	Deploy ML model endpoint	MVP Completion & Deployment	TO DO	Medium
SCRUM-70	Set up monitoring and error tracking	MVP Completion & Deployment	TO DO	Medium
SCRUM-71	Final end-to-end testing in production	MVP Completion & Deployment	TO DO	Medium
SCRUM-72	Create privacy policy page	MVP Completion & Deployment	TO DO	Medium
SCRUM-73	Finalize technical paper (incorporate feedback)	MVP Completion & Deployment	TO DO	Medium
SCRUM-74	Write deployment manual (step-by-step)	MVP Completion & Deployment	TO DO	Medium
SCRUM-75	Write user manual with screenshots	MVP Completion & Deployment	TO DO	Medium
SCRUM-76	Record final MVP demo video (5-10 min)	MVP Completion & Deployment	TO DO	Medium
SCRUM-77	Prepare final presentation slides	MVP Completion & Deployment	TO DO	Medium
SCRUM-87	As a research coordinator, I want to export prediction results so that I can use them in research analysis	AI Model Integration	TO DO	Medium
SCRUM-88	As a learner, I want to see visual cues related to severity so that I can connect imaging features to outcomes	AI Model Integration	TO DO	Medium



Test Cases

Test Summary



Module	Total	Passed	Failed	Coverage
Backend — imageProcessor.js	11	11	0	~95%
Backend — db.js	6	6	0	~90%
Backend — API routes (index.js)	52	52	0	~90%
Frontend — LoginForm	10	10	0	~92%
Frontend — RegisterForm	10	10	0	~91%
Frontend — ImageGallery	8	8	0	~90%
Frontend — ImageUploader	8	8	0	~85%
Frontend — Pages	16	16	0	~88%
TOTAL	121	121	0	~89%

Stack==> Node.js | Express | MongoDB | React | TypeScript | Jest | Vitest

Test Cases - Backend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
1	Returns a Buffer	1. Create a 100×100 RGB PNG buffer using sharp. 2. Call preprocessImage(buffer). 3. Inspect the returned value.	preprocessImage() output is a Buffer instance	✓ PASS
2	Outputs PNG by default	1. Call preprocessImage(buffer) with no format option. 2. Inspect output metadata using sharp(result).metadata().	Default format produces image/png metadata	✓ PASS
3	Converts image to grayscale	1. Create a colour PNG buffer. 2. Call preprocessImage(buffer). 3. Read output metadata with sharp.	Output has 1 channel (grayscale)	✓ PASS
4	No upscale for small images	1. Call preprocessImage(buffer, { width: 1024, height: 1024 }). 2. Read output metadata.	50x50 image stays ≤50px when target is 1024x1024	✓ PASS
5	Resizes large image to fit target	1. Call preprocessImage(buffer, { width: 512, height: 512 }). 2. Read output metadata.	2000x2000 image resized to ≤512x512 when target is 512	✓ PASS
6	Preserves aspect ratio	1. Call preprocessImage(buffer, { width: 200, height: 200 }). 2. Compute width/height ratio of output.	200x100 (2:1) input produces ~2:1 ratio output	✓ PASS
7	Outputs JPEG when format=jpeg	1. Call preprocessImage(buffer, { format: 'jpeg' }). 2. Read output metadata.	format:'jpeg' option produces jpeg metadata	✓ PASS
8	Outputs WebP when format=webp	1. Call preprocessImage(buffer, { format: 'webp' }). 2. Read output metadata.	format:'webp' option produces webp metadata	✓ PASS
9	Normalize option does not throw	1. Call preprocessImage(buffer, { normalize: true }). 2. Await the result.	normalize:true runs without error	✓ PASS
10	Throws on invalid buffer	1. Create Buffer.from('not-an-image'). 2. Call preprocessImage(badBuffer). 3. Await and expect rejection.	Non-image buffer rejects with 'Image preprocessing failed'	✓ PASS
11	Throws on null input	1. Call preprocessImage(null). 2. Await and expect rejection.	null input causes rejection	✓ PASS
12	Throws when MONGODB_URI missing	1. Delete process.env.MONGODB_URI. 2. Call connectToMongo(). 3. Await the promise.	connectToMongo() rejects with 'MONGODB_URI is not set'	✓ PASS
13	Connects and returns db instance	1. Set process.env.MONGODB_URI to a valid URI. 2. Call connectToMongo(). 3. Inspect return value.	Valid URI resolves to a db object	✓ PASS
14	Creates unique index on users.email	1. Call connectToMongo(). 2. Check that db.collection('users').createIndex was called.	createIndex called with {email:1}, {unique:true}	✓ PASS
15	Caches db on repeated calls	1. Call connectToMongo() twice. 2. Check client.connect call count.	client.connect called exactly once after two connectToMongo() calls	✓ PASS

Test Cases - Backend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
16	getDb throws before connect	1. Import getDb from fresh module. 2. Call getDb() immediately.	getDb() throws 'MongoDB not connected' before connectToMongo()	✓ PASS
17	getDb returns db after connect	1. Call connectToMongo(). 2. Call getDb(). 3. Inspect return value.	getDb() returns db object after successful connect	✓ PASS
18	GET /health returns 200	1. Send GET request to /health.	Response body is {status:'ok'}	✓ PASS
19	Register — valid input returns 201	1. POST /api/v1/auth/register with { email: 'test@example.com', password: 'password123' }. 2. Inspect response.	Returns id, email, createdAt on valid body	✓ PASS
20	Register — missing email → 400	1. POST /api/v1/auth/register with { password: 'password123' } only.	Returns 400 with 'required' message	✓ PASS
21	Register — missing password → 400	1. POST /api/v1/auth/register with { email: 'test@example.com' } only.	Returns 400	✓ PASS
22	Register — invalid email → 400	1. POST /api/v1/auth/register with { email: 'not-an-email', password: 'password123' }.	Returns 400 with 'Invalid email format'	✓ PASS
23	Register — short password → 400	1. POST /api/v1/auth/register with { email: 'test@example.com', password: 'short' }.	Returns 400 with '8 characters' message	✓ PASS
24	Register — duplicate user → 409	1. POST /api/v1/auth/register with existing email.	Returns 409 with 'User already exists'	✓ PASS
25	Register — email normalized	1. POST with { email: 'USER@EXAMPLE.COM', password: 'password123' }. 2. Check response body email field.	USER@EXAMPLE.COM stored as user@example.com	✓ PASS
26	Login — valid credentials → 200	1. POST /api/v1/auth/login with correct email and password.	Returns JWT token and user object	✓ PASS
27	Login — empty body → 400	1. POST /api/v1/auth/login with empty body {}.	Returns 400	✓ PASS
28	Login — unknown user → 401	1. POST /api/v1/auth/login with unregistered email.	Returns 401 with 'Invalid email or password'	✓ PASS
29	Login — wrong password → 401	1. POST /api/v1/auth/login with correct email but wrong password.	Returns 401	✓ PASS
30	Presign — valid input → 200	1. POST /api/v1/uploads/presign with { filename: 'xray.png', contentType: 'image/png' }.	Returns uploadUrl, fileUrl, method:PUT	✓ PASS

Test Cases - Backend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
31	Presign — missing filename → 400	1. POST /api/v1/uploads/presign with { contentType: 'image/png' } only.	Returns 400	✓ PASS
32	Presign — missing contentType → 400	1. POST /api/v1/uploads/presign with { filename: 'xray.png' } only.	Returns 400	✓ PASS
33	Images — valid input → 201	1. POST /api/v1/images with { fileUrl: '/uploads/test.png', originalName: 'xray.png', contentType: 'image/png' }.	Returns id, fileUrl, originalName, contentType	✓ PASS
34	Images — missing fileUrl → 400	1. POST /api/v1/images with { originalName: 'xray.png' } only.	Returns 400 with 'fileUrl is required'	✓ PASS
35	GET /images — no token → 401	1. Send GET /api/v1/images with no Authorization header.	Returns 401 with 'Authorization token is required'	✓ PASS
36	GET /images — malformed token → 401	1. Send GET /api/v1/images with Authorization: Bearer not.a.real.token.	Returns 401 with 'Invalid or expired token'	✓ PASS
37	GET /images — empty gallery → 200 []	1. mockFind returns []. 2. Send GET /api/v1/images with valid Bearer token.	Returns 200 with empty array []	✓ PASS
38	GET /images — returns images mapped correctly	1. mockFind returns 2 fake image docs. 2. Send GET /api/v1/images with valid Bearer token.	Returns 200, body length 2, first item has id 'img001' and fileUrl '/uploads/scan.png'	✓ PASS
39	GET /images — DB error → 500	1. mockFind rejects with Error('DB error'). 2. Send GET /api/v1/images with valid Bearer token.	Returns 500 with 'Failed to fetch images'	✓ PASS
40	POST /predict — no token → 401	1. POST /api/v1/predict with body { fileUrl: '/uploads/scan.png' } and no token.	Returns 401	✓ PASS
41	POST /predict — missing fileUrl → 400	1. POST /api/v1/predict with valid token and empty body {}.	Returns 400 with 'fileUrl is required'	✓ PASS
42	POST /predict — invalid fileUrl prefix → 400	1. POST /api/v1/predict with { fileUrl: '/etc/passwd' }.	Returns 400 with 'Invalid fileUrl'	✓ PASS
43	POST /predict — file not on disk → 404	1. mockFsAccess rejects (ENOENT). 2. POST /api/v1/predict with { fileUrl: '/uploads/scan.png' }.	Returns 404 with 'Uploaded file not found'	✓ PASS
44	POST /predict — image not in DB → 404	1. mockFsAccess resolves. mockCallMIPredict resolves. mockFindOne returns null. 2. POST /api/v1/predict.	Returns 404 with 'Image not found'	✓ PASS
45	POST /predict — returns cached prediction	1. mockFindOne returns imageDoc then existing prediction. 2. POST /api/v1/predict. 3. Check mockInsertOne was not called.	Returns 200, id='pred999', klGrade=1. mockInsertOne called 0 times	✓ PASS

Test Cases - Backend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
46	POST /predict — saves new prediction and returns ML result	1. mockFindOne returns imageDoc then null. 2. mockCallMIPredict returns {klGrade:2, confidence:0.87, label:'Moderate'}. 3. mockInsertOne returns insertedId 'pred001'. 4. POST /api/v1/predict.	Returns 200 with {id:'pred001', imageId:'img001', klGrade:2, confidence:0.87}. mockInsertOne called once	✓ PASS
47	POST /predict — ML service error → 500	1. mockFsAccess resolves. 2. mockCallMIPredict throws Error('ML service down'). 3. POST /api/v1/predict.	Returns 500 with 'Prediction failed'	✓ PASS
48	GET /predictions — no token → 401	1. Send GET /api/v1/predictions with no Authorization header.	Returns 401	✓ PASS
49	GET /predictions — invalid token → 401	1. Send GET /api/v1/predictions with Authorization: Bearer invalid.token.here.	Returns 401	✓ PASS
50	GET /predictions — empty list → 200 []	1. mockFind returns []. 2. Send GET /api/v1/predictions with valid Bearer token.	Returns 200 with empty array []	✓ PASS
51	GET /predictions — returns predictions mapped correctly	1. mockFind returns 2 fake prediction docs. 2. Send GET /api/v1/predictions with valid Bearer token.	Returns 200, body length 2. First item: id='pred001', result.klGrade=2. Second item: result.klGrade=0	✓ PASS
52	GET /predictions — DB error → 500	1. mockFind rejects with Error('DB connection lost'). 2. Send GET /api/v1/predictions with valid Bearer token.	Returns 500 with 'Failed to fetch predictions'	✓ PASS

Test Cases - Frontend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
1	Renders all form fields	1. Render <LoginForm onLoginSuccess={fn} />. 2. Query for email input, password input, Login button.	Email, password inputs and Login button present	✓ PASS
2	Empty email shows error	1. Click the Login button without filling any field.	Submitting with no email shows 'Email is required'	✓ PASS
3	Empty password shows error	1. Type a valid email. 2. Click Login without filling password.	Submitting with no password shows 'Password is required'	✓ PASS
4	No fetch on validation failure	1. Click Login with empty fields.	fetch() not called when form is invalid	✓ PASS
5	Fetch called with correct payload	1. Type 'User@Example.COM' in email field. 2. Type 'password123' in password field. 3. Click Login.	Email lowercased, sent to /api/v1/auth/login	✓ PASS
6	Token stored in localStorage	1. Submit valid credentials. 2. Check localStorage after submission.	localStorage.getItem('token') equals returned JWT	✓ PASS
7	onLoginSuccess called on success	1. Submit valid credentials. 2. Wait for async resolution.	Callback fires once after successful login	✓ PASS
8	Server error shown	1. Submit credentials. 2. Wait for response.	'Invalid email or password' shown on 401 response	✓ PASS
9	Network error shown	1. Submit valid credentials.	'Network Error' shown when fetch throws	✓ PASS
10	Button disabled while loading	1. Submit valid credentials. 2. Immediately inspect button state.	Submit button is disabled while fetch is pending	✓ PASS
11	Renders all form fields	1. Render <RegisterForm onRegistrationSuccess={fn} />. 2. Query for email, password, confirm password, Register button.	Email, password, confirm password, Register button present	✓ PASS
12	Empty email shows error	1. Click Register without filling any field.	'Email is required' shown on empty submit	✓ PASS
13	Short password shows error	1. Type email. 2. Type password shorter than 8 characters. 3. Click Register.	'at least 8 characters' shown for short password	✓ PASS
14	Password mismatch shows error	1. Fill email and password. 2. Type different value in confirm password. 3. Click Register.	'Passwords do not match' shown when fields differ	✓ PASS
15	No fetch on validation failure	1. Click Register with empty form.	fetch() not called when form is invalid	✓ PASS

Test Cases - Frontend



Test Case	Test Name	Test Steps	Description / Expected Result	Status
16	Fetch called with correct payload	1. Fill 'User@Example.COM', 'password123', 'password123'. 2. Click Register.	Email lowercased, sent to /api/v1/auth/register	✓ PASS
17	Success message and redirect	1. Submit valid form. 2. Advance timers by 800ms.	'Registration successful' shown; onRegistrationSuccess fires after 800ms	✓ PASS
18	Form cleared on success	1. Submit valid form. 2. Wait for success state.	All fields empty after successful registration	✓ PASS
19	Server error shown	1. Submit valid form.	'User already exists' shown on 409 response	✓ PASS
20	Button disabled while submitting	1. Submit valid form. 2. Inspect button state.	Register button disabled while fetch pending	✓ PASS
21	Empty state message shown	1. Render <ImageGallery images={} />.	'No images to display' shown for empty array	✓ PASS
22	Empty state for undefined prop	1. Render <ImageGallery images={undefined} />.	'No images to display' shown when images is undefined	✓ PASS
23	Renders img per image	1. Render <ImageGallery images={[img1, img2]} />. 2. Query all elements.	One element rendered per image with url	✓ PASS
24	title used as alt text	1. Render gallery with image { url: '...', title: 'Knee X-ray' }.	img alt equals image.title	✓ PASS
25	Falls back to originalName	1. Render gallery with image { url: '...', originalName: 'scan.png' }.	img alt equals originalName when title absent	✓ PASS
26	Falls back to fileUrl	1. Render gallery with image { fileUrl: '/uploads/scan.png', title: 'Scan' }.	img src equals fileUrl when url absent	✓ PASS
27	No preview placeholder	1. Render gallery with image { id: '1', title: 'Mystery' }.	'No preview' shown when url and fileUrl both absent	✓ PASS
28	Label text rendered below image	1. Render gallery with image { url: '...', title: 'Knee Scan' }.	Title/name text visible below the image	✓ PASS
29	Renders dropzone text	1. Render <ImageUploader />.	'Drag & drop or click' text visible	✓ PASS
30	Shows drag-active text	1. Fire dragOver event on the dropzone element.	'Drop files here' shown on dragenter	✓ PASS

Test Cases - Frontend



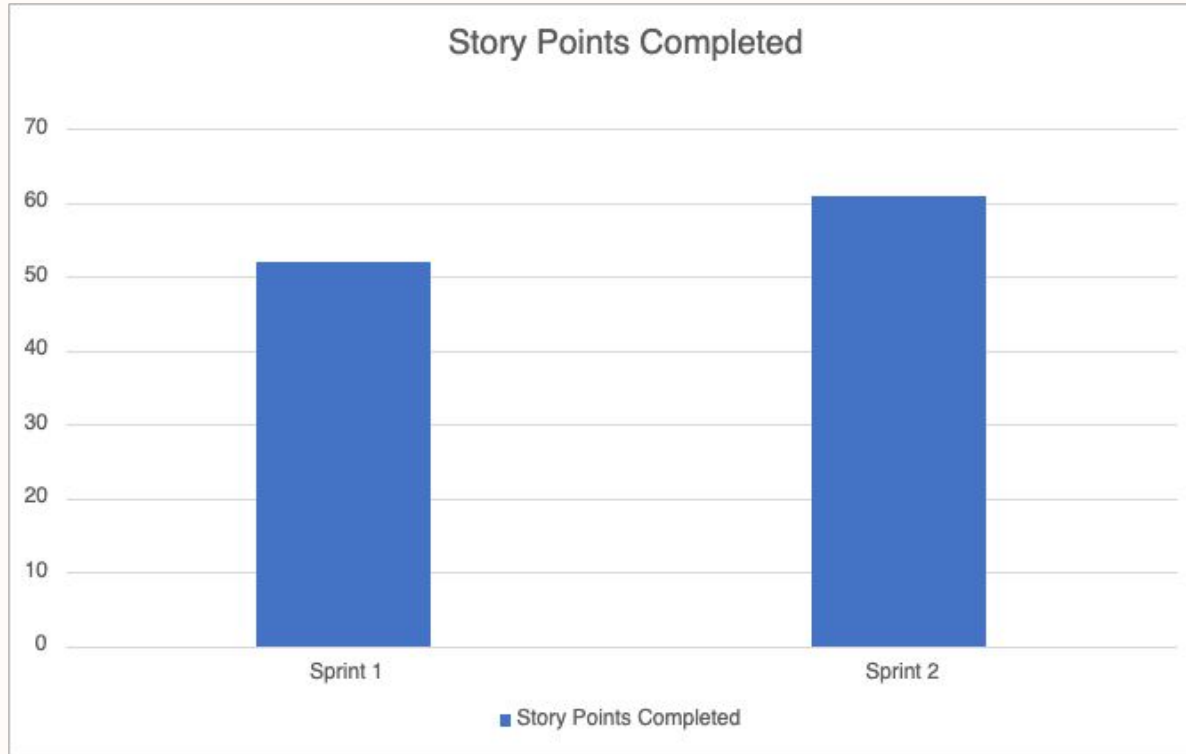
Test Case	Test Name	Test Steps	Description / Expected Result	Status
31	No upload button initially	1. Render <ImageUploader /> without dropping files.	Upload button absent before files are dropped	✓ PASS
32	Upload button after drop	1. Fire drop event with a PNG file. 2. Query for Upload button.	Upload button appears after files are dropped	✓ PASS
33	DICOM placeholder shown	1. Fire drop event with a .dcm file. 2. Query for DICOM label.	'DICOM' placeholder shown for .dcm files	✓ PASS
34	Full upload flow succeeds	1. Drop a PNG file. 2. Click Upload. 3. Wait for all three fetch calls.	presign → PUT → register called in order; onUploadSuccess fires	✓ PASS
35	Alert on presign failure	1. Drop a file. 2. Click Upload.	window.alert called when presign returns non-ok	✓ PASS
36	Button disabled while uploading	1. Drop a file. 2. Click Upload. 3. Inspect button state.	Upload button disabled while fetch is pending	✓ PASS
37	LoginPage — brand heading	1. Render <LoginPage onLoginSuccess={fn} onSwitchToRegister={fn} />.	'KneeVision' h1 rendered	✓ PASS
38	LoginPage — subtitle	1. Render LoginPage.	'Sign in to your account' text present	✓ PASS
39	LoginPage — form rendered	1. Render LoginPage.	Email and password inputs present inside page	✓ PASS
40	LoginPage — switch to register	1. Click the Register button.	onSwitchToRegister called when Register button clicked	✓ PASS
41	RegisterPage — brand heading	1. Render <RegisterPage onRegistrationSuccess={fn} onSwitchToLogin={fn} />.	'KneeVision' h1 rendered	✓ PASS
42	RegisterPage — subtitle	1. Render RegisterPage.	'Create your account' text present	✓ PASS
43	RegisterPage — form rendered	1. Render RegisterPage.	Email and confirm password inputs present inside page	✓ PASS
44	RegisterPage — switch to login	1. Click the Login button.	onSwitchToLogin called when Login button clicked	✓ PASS
45	UploadPage — brand heading	1. Render <UploadPage onLogout={fn} />.	'KneeVision' h1 rendered	✓ PASS

Test Cases - Frontend



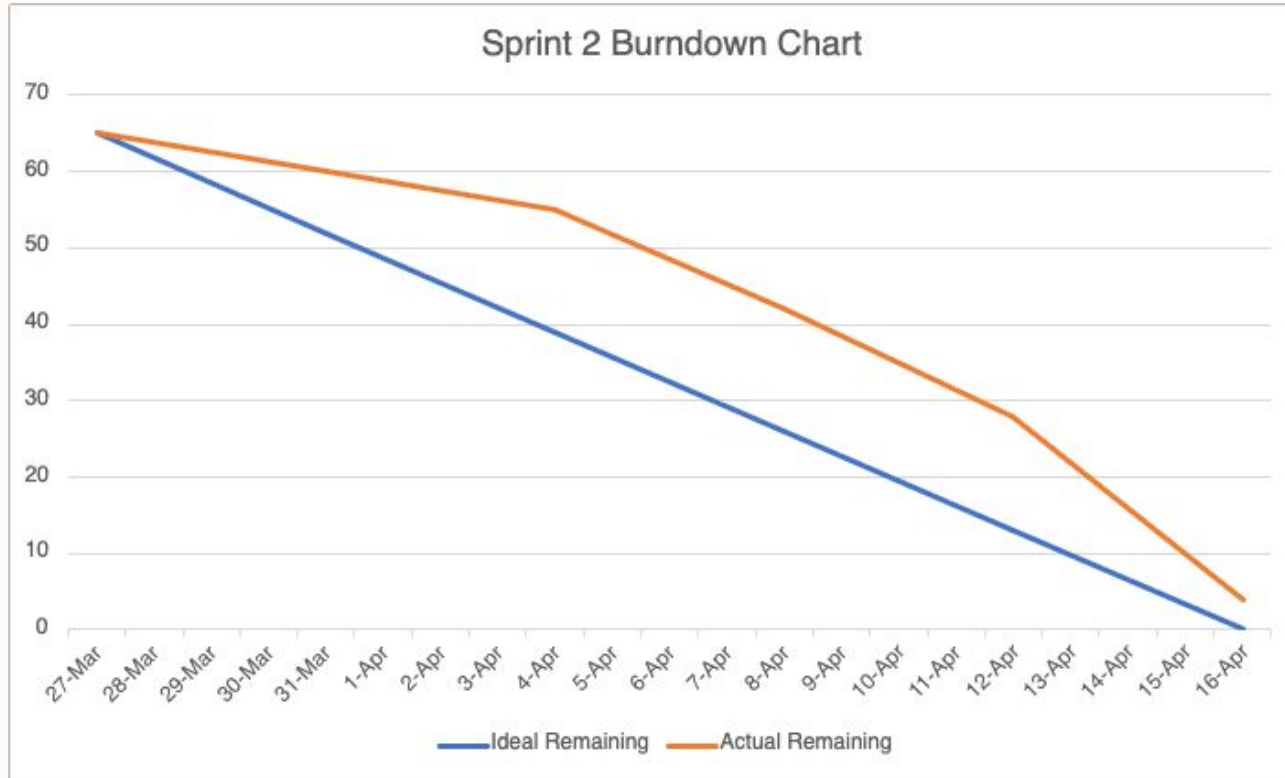
Test Case	Test Name	Test Steps	Description / Expected Result	Status
46	UploadPage — subtitle	1. Render UploadPage.	'Upload and review knee X-ray' text present	✓ PASS
47	UploadPage — logout button	1. Render UploadPage.	Logout button visible	✓ PASS
48	UploadPage — logout callback	1. Click the Logout button.	onLogout called when Logout clicked	✓ PASS
49	UploadPage — gallery heading	1. Render UploadPage.	'Gallery' heading present	✓ PASS
50	UploadPage — empty state	1. Render UploadPage with no images.	'No images to display yet' shown initially	✓ PASS
51	UploadPage — add mock image	1. Click 'Add Mock Image'. 2. Query for img element.	Mock image appears in gallery after clicking 'Add Mock Image'	✓ PASS
52	UploadPage — empty state hidden	1. Click 'Add Mock Image'. 2. Query for empty state text.	Empty state message gone after image added	✓ PASS

Metrics - Team Velocity



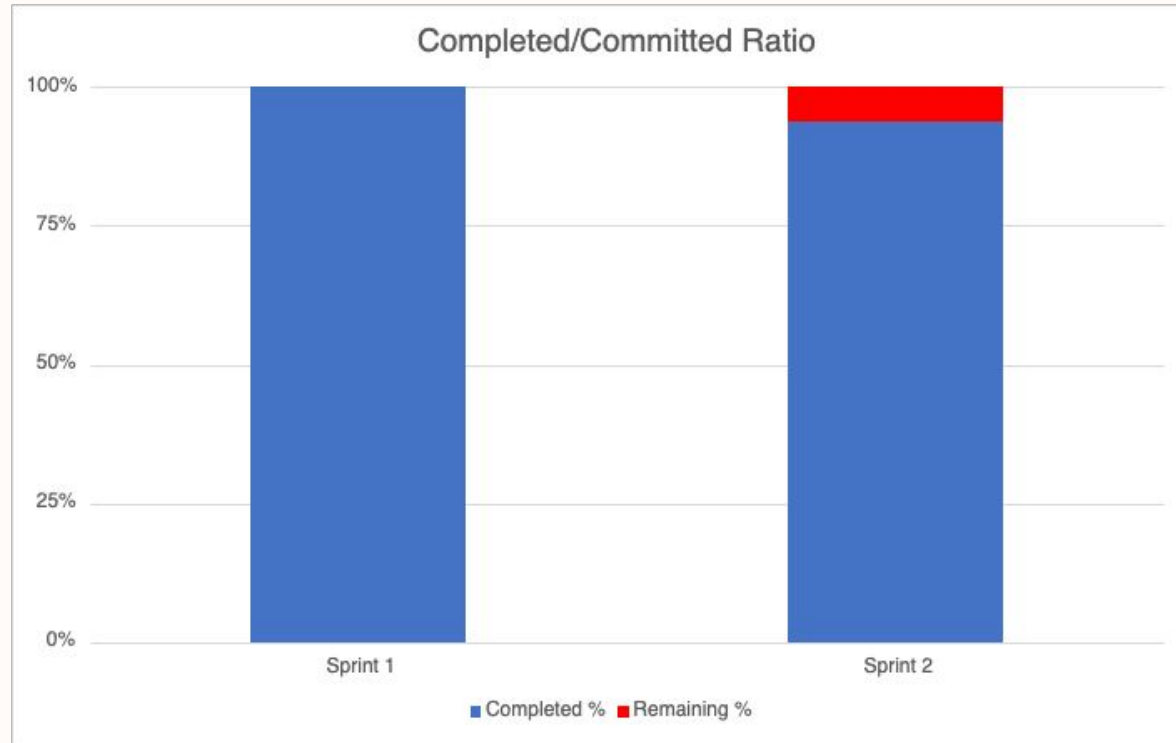
Sprint 2 Story Points Completed: **61**

Metrics - Burndown Chart



Sprint 2 Story Points Remaining: 4

Metrics - Completed/Committed Ratio



Sprint 2 Story Points Completed: **61/65 (93.85%)**

Sprint 2 Retrospective

KneeVision - Sprint 2 Retrospective

What went well +

ML prediction flow fully integrated (upload → predict → store → retrieve) + 5	Dashboard implementation improved overall UX (clear landing after login) + 4
History page enabled full visibility into past uploads and predictions + 4	Strong frontend-backend integration (images ↔ predictions mapping) + 3
Team adapted quickly to new features (filters, sorting, navigation) + 4	Issues were debugged efficiently (Docker, model path, environment setup) + 3

What can be improved +

Initial feature integration caused some merge conflicts and rework + 4	Some inconsistencies in data flow (images vs predictions linking) + 5
Limited early planning for features like history and dashboard + 4	UI/UX color scheme and visual consistency could be improved + 4
Feature scope was not always clearly defined before implementation + 5	Some components required refactoring after initial implementation + 7

Action Items +

Decide on all AI/ML features to be implemented moving forward + 7	Define data contracts early (images, predictions, dashboard metrics) + 4
Improve PR review turnaround and consistency across team + 3	Standardize navigation across all pages (Dashboard / Upload / History) + 3
Improve documentation for local setup (Docker + model handling) + 3	Plan feature scope more clearly before starting development + 6

Sprint 2 Retrospective: Important Points



What Went Well

- ML prediction flow fully integrated
- Dashboard implementation improved overall UX
- History page enabled full visibility into past uploads and predictions
- Team adapted quickly to new features



Improvements

- Some inconsistencies in data flow
- Feature scope was not always clearly defined before implementation
- Some components required refactoring after initial implementation



Action Items

- Decide on all AI/ML features to be implemented moving forward
- Plan feature scope more clearly before starting development

Project Roadmap

Sprint Schedule & Key Deliverable

0

DEADLINE: FEB 19

Foundation

Project Charter & MVP
GitHub & Tech Stack Setup
Backlog Creation

DELIVERABLES:

Presentation + Wiki + Backlog

1

DEADLINE: MAR 26

Core Features

User Auth
Image Upload
Image Gallery

DELIVERABLES:

Working Demo + Wiki + Tests

2

DEADLINE: APR 16

AI Engine

Model Training
Prediction API
KL Grade Results

DELIVERABLES:

AI Demo + Draft Paper + Wiki

3

DEADLINE: MAY 14

Deployment

History & CSV Export
Security & UAT Audit
Production Launch

DELIVERABLES:

Final MVP + Full Documentation

SPRINT 3 - USER STORIES

US_ID	User Stories	Acceptance Criteria	Feature	Story Points
US_03	As an individual user, I want to register an account so that I can store my basic information and access personalised services.	The user can access a registration page requiring: Full name, Email address, Password creation. Post-registration, the user can log in and update their profile information.	User Management	2
US_04	As an individual user, I want to log in to my account so that I can manage my cases and view diagnoses.	The user accesses a login page requiring: Email address, Password. Upon successful login, the user is directed to their dashboard.	User Management	1
US_05	As a user, I want to view my past cases and track their statuses so that I can monitor my health progress.	The user can access a 'Case History' section after logging in. All past cases are listed in chronological order (most recent first). Each case displays: case ID, case open date, status. The system updates case statuses in real-time. Clicking on a case entry navigates the user to that case's report.	User Management	2
US_06	As an individual user, I want to implement JWT authentication so that my session is secure.	JWT tokens are generated on login and validated on every protected route. Tokens expire after a defined period and require re-authentication. Invalid or expired tokens return a 401 Unauthorized response.	User Management	3

Project - APIs - Login API

```
app.post("/api/v1/auth/login", async (req, res) => {
  const { email, password } = req.body || {};
  const normalizedEmail = String(email || "").trim().toLowerCase();

  const db = getDb();
  const users = db.collection("users");
  const user = await users.findOne({ email: normalizedEmail });

  if (!user) {
    return res.status(401).json({ message: "Invalid email or password." });
  }

  const passwordMatches = await bcrypt.compare(password, user.passwordHash);
  if (!passwordMatches) {
    return res.status(401).json({ message: "Invalid email or password." });
  }

  const token = jwt.sign(
    { userId: user.userId || user._id.toString(), email: user.email },
    process.env.JWT_SECRET || "dev-secret-change-me",
    { expiresIn: "1h" }
  );

  return res.status(200).json({ token, user: { userId: user.userId, email: user.email } });
});
```

Project - APIs - Upload Image API

```
app.post("/api/v1/images", requireAuth, async (req, res) => {
  const { fileUrl, originalName, contentType } = req.body || {};

  const imageDoc = {
    fileUrl,
    originalName: originalName || null,
    contentType: contentType || null,
    uploadedBy: {
      userId: req.user.userId,
      email: req.user.email,
    },
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
  };

  const db = getDb();
  const images = db.collection("images");
  const result = await images.insertOne(imageDoc);

  return res.status(201).json({
    id: result.insertedId.toString(),
    fileUrl: imageDoc.fileUrl,
    uploadedBy: imageDoc.uploadedBy,
  });
});
```

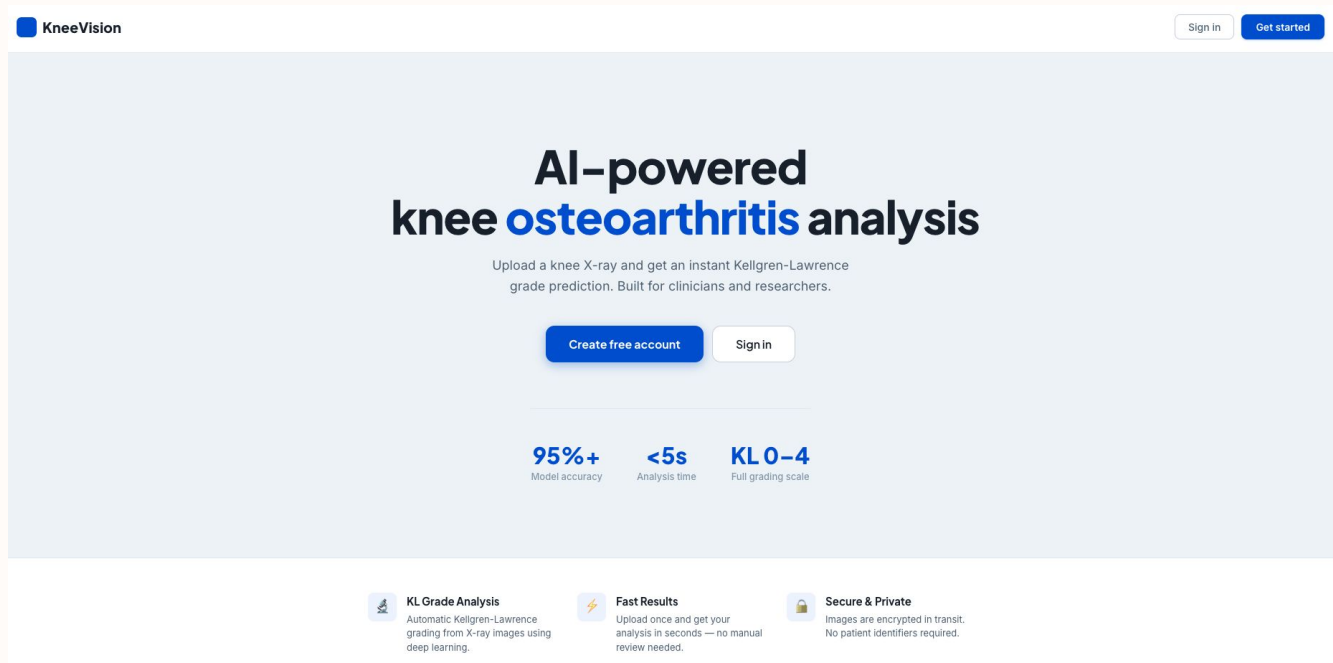
Project - APIs - Get Images API

```
app.get("/api/v1/images", requireAuth, async (req, res) => {
  const db = getDb();
  const images = db.collection("images");

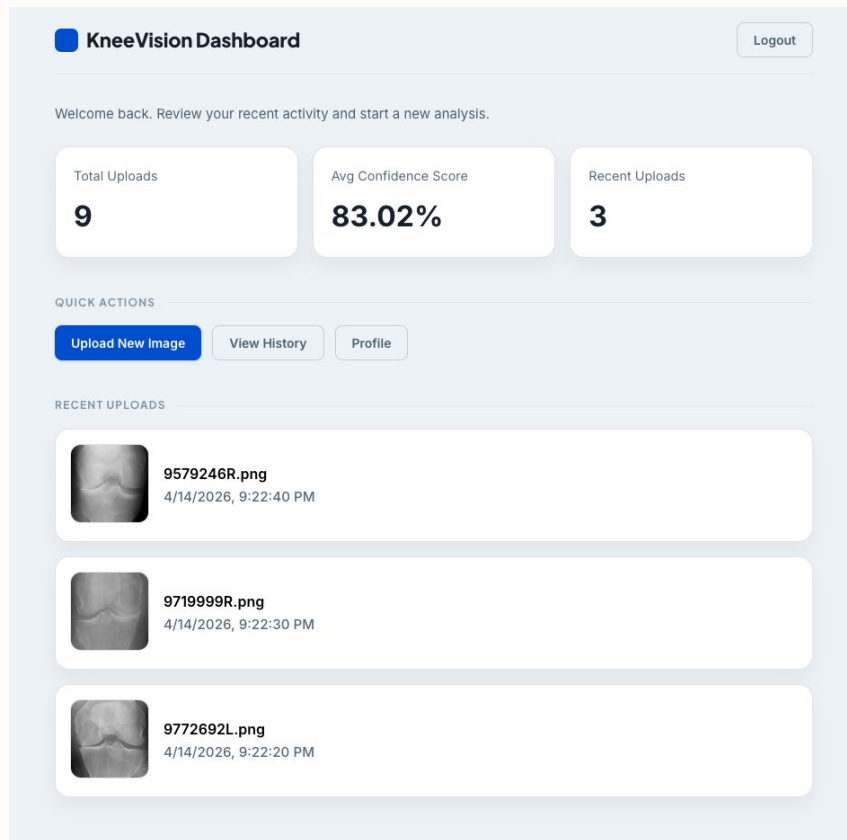
  const docs = await images
    .find({ "uploadedBy.userId": req.user.userId })
    .sort({ createdAt: -1 })
    .toArray();

  return res.status(200).json(
    docs.map((doc) => ({
      id: doc._id.toString(),
      fileUrl: doc.fileUrl,
      originalName: doc.originalName,
      uploadedBy: doc.uploadedBy,
      createdAt: doc.createdAt,
    }))
  );
});
```

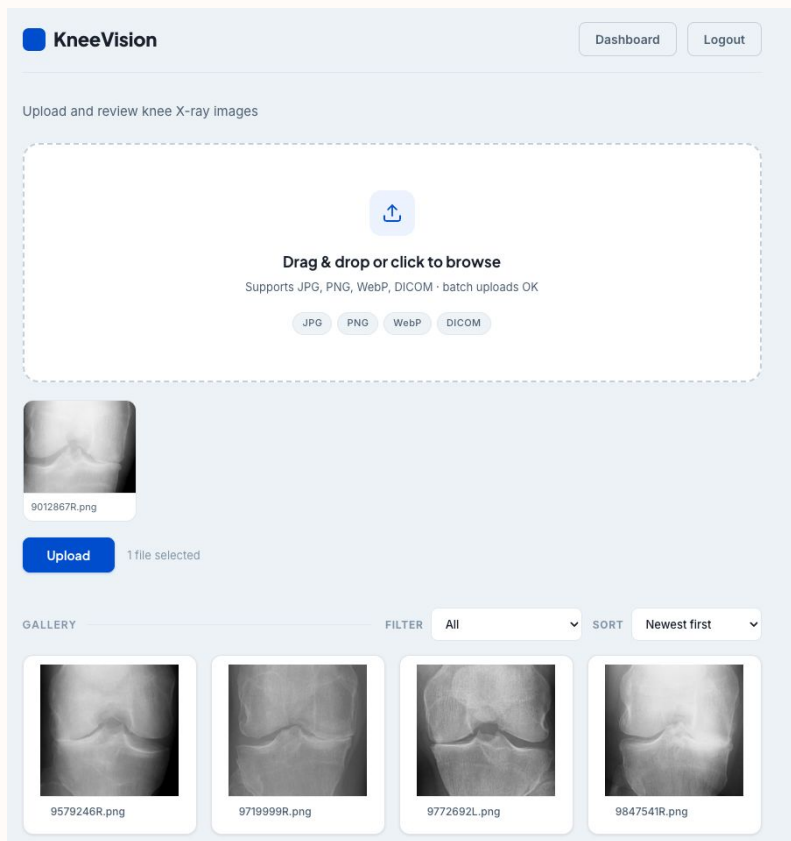
Project Demo - Application Screenshots



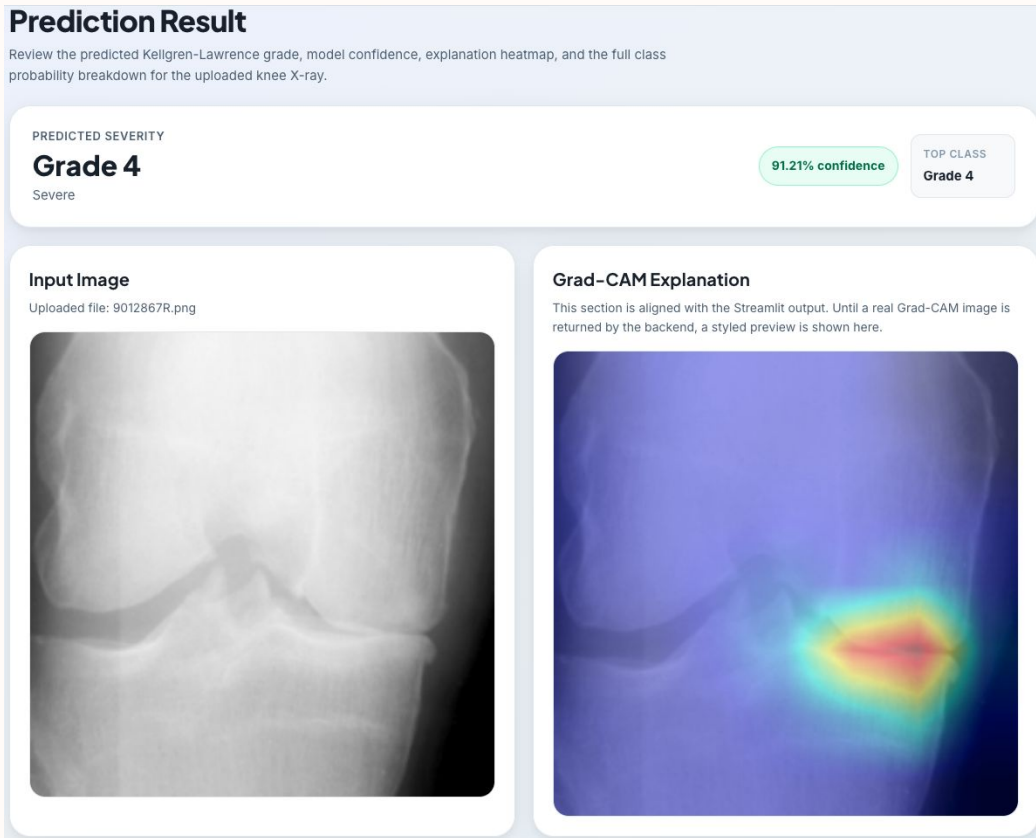
Project Demo - Application Screenshots



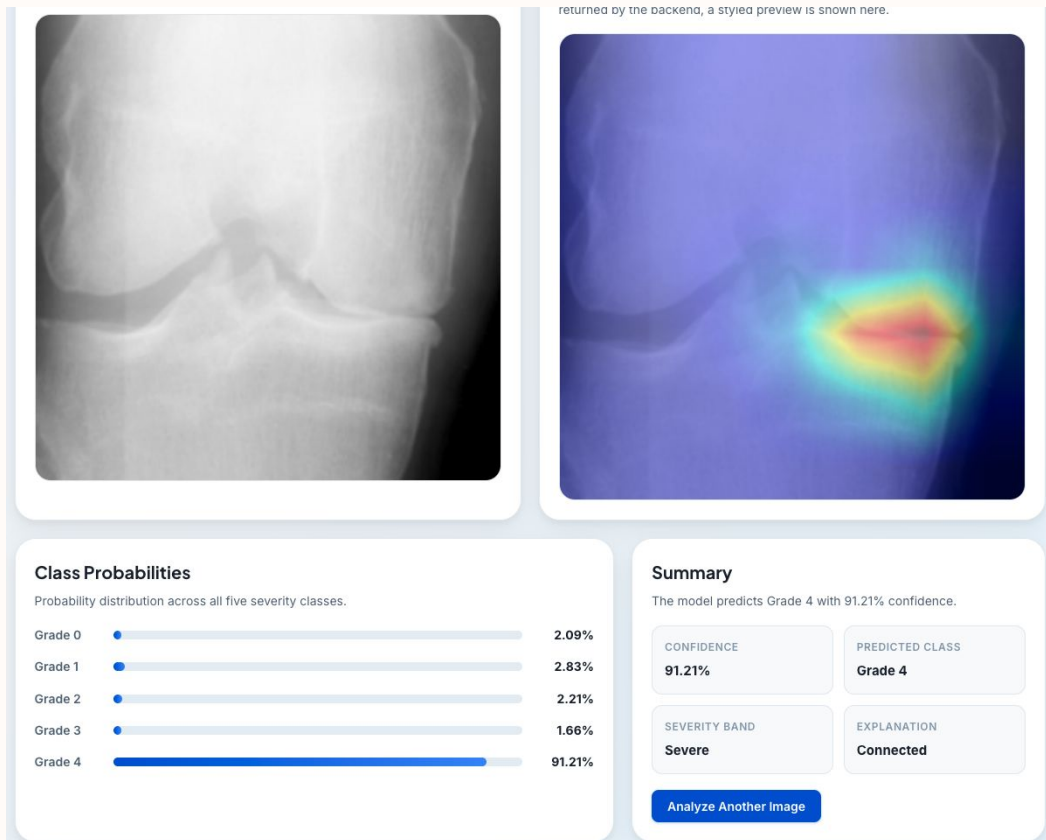
Project Demo - Application Screenshots



Project Demo - Application Screenshots



Project Demo - Application Screenshots



Project Demo - Application Screenshots

 **KneeVision History**

Dashboard

Upload

Logout

Browse past uploads and reopen saved analysis results.

ANALYSIS HISTORY

FILTER

All

GRADE

All Grades

SORT

Newest first



9012867R.png
4/14/2026, 9:25:22 PM

Grade 4

91.21%

Severe



9579246R.png
4/14/2026, 9:22:40 PM

Grade 4

90.14%

Severe



9719999R.png
4/14/2026, 9:22:30 PM

Grade 4

94.72%

Severe

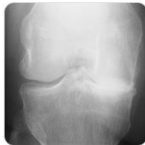


9772692L.png
4/14/2026, 9:22:20 PM

Grade 4

80.86%

Severe



9847541R.png
4/14/2026, 9:22:09 PM

Grade 4

91.85%

Severe

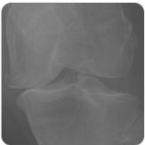


9924274L.png
4/14/2026, 9:21:47 PM

Grade 4

94.61%

Severe



9964731L.png
4/14/2026, 9:21:32 PM

Grade 4

86.24%

Severe



9992318L.png
4/14/2026, 9:21:20 PM

Grade 4

87.80%

Severe

Wiki Page Link

<https://github.com/htmhw/STARLABS/wiki#kneevision>

Docker:

Containers [Give feedback](#)

View all your running containers and applications. [Learn more](#)

Container CPU usage ⓘ

0.64% / 1200% (12 CPUs available)

Container memory usage ⓘ

606.64MB / 7.44GB

[Show charts](#)



Only show running containers

☐	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☐ ▾ ●	infra	-	-	-	0.64%	43 minutes ago	
☐ ●	ml-service-1	90d81254ea8b	infra-ml-service	8000:8000 ↗	0.34%	43 minutes ago	
☐ ●	backend-1	81a2925d5724	infra-backend	4000:4000 ↗	0.3%	43 minutes ago	
☐ ●	frontend-1	d0a7fbdeffc1	infra-frontend	80:80 ↗	0%	43 minutes ago	

Live Application Demo



AI Disclosure

- Assisted in generating initial user stories and backlog items
- Helped draft test cases and testing scenarios
- Supported development of specific backend API routes (authentication, image upload, and database integration) and frontend components (forms, state handling, and API calls)
- Provided guidance for debugging and resolving errors
- Assisted with structuring documentation and project artifacts



THANK YOU

Team S.T.A.R Labs